

WEEK 11 · ASYNC, PROMISES, FETCH

THE WORLD OUTSIDE IS SLOW

Last week the data stopped living in the tab.

Today it stops living on your machine.

WHERE WE ARE

Five files, one loop, data that survives a reload — and every byte of it typed by you.

AFTER TODAY YOU CAN

Read from somebody else's computer, and stay honest with your user when it is slow, wrong, or not there.

WHAT THIS UNLOCKS

The two UI states week 6 promised and nothing has needed yet: **loading** and **error**.

JavaScript has one thread.

`await` does not add one.

EVERYTHING ON YOUR SCREEN WAS TYPED BY YOU

WHAT WE HAVE

Six groceries in `items.js`, plus whatever the user added. Correct, persistent, and entirely invented.

WHAT A REAL APP HAS

A catalogue somebody else maintains, on a machine you do not control, over a network you cannot see.

AND WHAT THAT COSTS

It is **slow**. It **fails**. It sometimes answers **something else**. All three are now your problem.

Modern web apps are data-driven: data comes from a server, is stored in the browser, and is exchanged in a standard format.

ONE THREAD, AND WHAT HOLDING IT COSTS

two seconds, three ways – and a counter that says whether the page is alive↶ reloadopen ↷

TWO SECONDS, THREE WAYS

block for 2s

await 2s

setTimeout 2s

reset

THE PAGE, WHILE IT HAPPENS

frames painted since you pressed: **0**

A healthy page paints about 60 of these a second. Two seconds should be ~120.

type here during the two seconds

Nothing typed yet.

WHAT ACTUALLY RAN

RUN

HOW

WALL CLOCK

FRAMES PAINTED

THE THREAD WAS

► Press **BLOCK FOR 2S** and try to type in the box while it runs. Then press **AWAIT 2S** and do the same.

THE SAME JOB, THREE WAYS TO WRITE IT

find the user, then the orders, then the total – measured↺ reloadopen ↗

FIND THE USER, THEN THE ORDERS, THEN THE TOTAL

run all three ☐ make step 2 fail Ready.

CALLBACKS

```
findUserCb('דנה', (err, user) => {
  if (err) return show(err);
  findOrdersCb(user.id, (err, orders) => {
    if (err) return show(err);
    findTotalCb(orders, (err, total) => {
      if (err) return show(err);
      show(total);
    });
  });
});
```

—

PROMISE CHAIN

```
findUser('דנה')
  .then((user) => findOrders(user.id))
  .then((orders) => findTotal(orders))
  .then((total) => show(total))
  .catch(show);
```

—

ASYNC / AWAIT

```
try {
  const user = await findUser('דנה');
  const orders = await findOrders(user.id);
  const total = await findTotal(orders);
  show(total);
} catch (err) {
  show(err);
}
```

—

VERSION	ANSWER	WALL CLOCK	DEEPEST INDENT	PLACES THE FAILURE IS HANDLED

► Run it once with the failure off. Same answer, same time. Then tick **MAKE STEP 2 FAIL**.

4

A PROMISE IS THREE STATES, AND IT SETTLES ONCE

```
1 const p = new Promise((resolve, reject) => {  
2   // pending until one of them is called  
3 });  
4  
5 p.then((value) => { /* fulfilled */ })  
6 .catch((error) => { /* rejected */ })  
7 .finally(() => { /* both paths */ });
```

1 pending
— nobody knows yet

2 fulfilled
— there is a value

3 rejected
— there is a reason

A settled promise never settles again.

try/catch WORKS AGAIN, AND THAT IS THE WHOLE GIFT

A CALLBACK CANNOT BE CAUGHT

```
1 try {  
2   load(url, (err, data) => {  
3     // by the time this runs, the try  
4     // below has ALREADY finished  
5   });  
6 } catch { /* never reached */ }
```

AN AWAIT CAN

```
1 try {  
2   const data = await load(url);  
3   show(data);  
4 } catch (error) {  
5   // a rejection ANYWHERE in the chain lands here  
6   showProblem(error);  
7 } finally {  
8   stopSpinner();           // both paths  
9 }
```

► `await` turns a rejection into a `throw` at exactly the line you were waiting on. That is the entire syntactic trick.

A REJECTION IS NOT AN EXCEPTION UNTIL SOMEBODY ASKS

the same rejected promise, caught now and caught a second later

↺ reload

open ↗

THE THREE STATES, AND THE FOURTH THING NOBODY MENTIONS

new promise

resolve it

reject it

state: —

WHAT RAN, IN ORDER

press "new promise"

THE CODE THAT IS ATTACHED

```
const p = new Promise((resolve, reject) => {  
  // nothing here settles it – the buttons do  
});  
  
p.then((value) => log('then: ' + value))  
  .catch((err) => log('catch: ' + err.message))  
  .finally(() => log('finally – on both paths'));
```

A REJECTION NOBODY IS LISTENING TO

reject, and catch it immediately

reject, and catch it one second later

unhandledrejection events so far: 0

CAUGHT IN THE SAME TICK

```
const p = Promise.reject(new Error('boom'));  
p.catch(handle);      // attached NOW
```

—

CAUGHT ONE SECOND LATER

```
const p = Promise.reject(new Error('boom'));  
setTimeout(() => p.catch(handle), 1000);
```

—

Press both. The promise is identical; only the moment the listener arrives differs.

► Press both buttons at the bottom. **THE PROMISE IS IDENTICAL** — only the moment the listener arrives differs.

`async` AND `await` ARE THE SAME THING, WRITTEN FLAT

WHAT PEOPLE THINK IT MEANS

```
1 // "await runs this on another thread
2 // and waits for it there"
3
4 const items = getItem(); // items is a Promise
```

WHAT IT MEANS

```
1 // "suspend THIS function and give the
2 // one thread back until the answer is here"
3
4 const items = await getItem();
5 // ↑ an async function ALWAYS returns a Promise
```

► `items.length` without the `await` is undefined — and that is the single most common mistake in this subject.

WHAT ORDER DOES THIS ACTUALLY RUN IN?

seven statements, and the order is not the order

reload

open

SEVEN STATEMENTS. WHAT ORDER DO THEY RUN IN?

run it

starve the timer

Guess first. Then press.

THE SOURCE, AS WRITTEN

```
console.log('1 - synchronous, top');           // line 1

setTimeout(() => console.log('timer'), 0);       // line 3

Promise.resolve().then(() => console.log('promise .then')); // line 5

queueMicrotask(() => console.log('queueMicrotask')); // line 7

(async () => {
  console.log('inside async, BEFORE the await'); // line 10
  await null;                                   // line 11
  console.log('inside async, AFTER the await');  // line 12
})();

console.log('2 - synchronous, bottom');         // line 15
```

THE ORDER IT RAN

-

#	LABEL	QUEUE	WRITTEN AT LINE	RAN AT POSITION
---	-------	-------	-----------------	-----------------

A TIMER THAT ASKED FOR ZERO

The same measurement four times, with a longer chain of promise callbacks each time. In every run a `setTimeout(fn, 0)` is scheduled **first**, and then the chain is built. Microtasks drain completely before the next macrotask, and a microtask that queues another microtask joins the same drain.

► TAKE A VOTE BEFORE YOU PRESS. Write the room's answer on the board.

FIVE REQUESTS, AND THE LOOP THAT COSTS YOU FIVE TIMES

CORRECT, AND FIVE TIMES SLOWER

```
for (const url of urls) {  
  results.push(await fetch(url));  
}  
// each one STARTS after the last one finished
```

CORRECT

```
const results = await Promise.all(  
  urls.map((url) => fetch(url)),  
);  
// all five start now; you wait for the slowest
```

► The loop is **RIGHT** when step 2 needs step 1's answer. These five do not.

ALL · ALLSETTLED · RACE, WITH A STOPWATCH

five tasks – 500, 300, 200, 400, 100 ms↻ reloadopen ↗

FIVE TASKS: 500, 300, 200, 400, 100 MS

await in a loop **Promise.all** Promise.allSettled Promise.race ☐ make task 3 fail reset

WHAT EACH ONE RETURNED
–

THE SHAPE OF EACH CALL
for (const t of tasks) out.push(await t()); // sum of all five

await Promise.all(tasks.map(t => t())); // slowest one
await Promise.allSettled(tasks.map(t => t())); // slowest one, never rejects
await Promise.race(tasks.map(t => t())); // fastest one to SETTLE
–

CALL	WALL CLOCK	VS THE THEORETICAL BEST	OUTCOME	RESULTS YOU CAN ACTUALLY USE
------	------------	-------------------------	---------	------------------------------

► Run the loop and `Promise.all` back to back. Then tick **MAKE TASK 3 FAIL** and run `all` and `allSettled`.

WHICH ONE, AND WHEN

`Promise.all`

I need all of them. Three panels of a dashboard. One fails and there is nothing worth drawing.

`Promise.allSettled`

I need what I can get. Six independent widgets. Five that loaded beat a blank page.

`Promise.race`

Whichever settles first. Its real use is a timeout — and note *settles*, so a fast failure wins.



THE WHOLE OF `fetch`, AND THEN THE HARD PART

```
1 const res = await fetch(url);           // a Response, not the data
2
3 if (!res.ok) { /* the server answered, and said no */ }
4
5 const data = await res.json();          // a SECOND await, a second failure
```

TWO LINES OF API

Send a request, read a body. That is the whole surface.

THREE LINES OF JUDGEMENT

Did it complete? Did the server agree? Was the body what you expected?

AND THE DEFAULT IS WRONG

No timeout. No status check. No shape check. **All three are yours.**

FETCH RESPONSE IS NOT THE DATA — THIS IS A CRITICAL CONCEPT

six responses through two loaders↻ reloadopen ↗

SIX RESPONSES, TWO LOADERS

1 · 200, good JSON 2 · 404, valid JSON 3 · 500, valid JSON 4 · 200, an HTML login page 5 · 200, empty body 6 · the network is gone

WHAT THE SERVER SENT

THE LOADER EVERYBODY WRITES

```
async function load(url) {
  const res = await fetch(url);
  const data = await res.json(); // no check of res.ok, ever
  return data.title;
}
```

—

THE LOADER IN THE ASSIGNMENT

```
async function load(url) {
  const res = await fetch(url); // rejects ONLY if it never completed
  if (!res.ok) throw new Error('HTTP ' + res.status); // <- the whole slide
  let data;
  try { data = await res.json(); } // a SECOND thing that can fail
  catch { throw new Error('the body was not JSON'); }
  if (typeof data?.title !== 'string') throw new Error('not the shape we expect');
  return data.title;
}
```

—

CASE	PROMISE REJECTED?	RES.OK	RES.STATUS	NAIVE LOADER
do it for real, over the network				

- Run case 1, then case 2, and ask what `try/catch` would have done. Write the answer on the board **BEFORE** running case 6.

COMMON MISTAKES

- 1 forgetting `return response.json()`
- 2 forgetting `await`
- 3 assuming `fetch` throws on 404 — **it doesn't**

A `fetch()` promise only rejects on network errors. It does not reject on HTTP error statuses like 404 or 500.

It rejects for one reason: the request never completed. DNS, no route, CORS, abort.

STATUS CODES, IN FOUR FAMILIES AND ONE QUESTION

2XX — IT WORKED

200, 201 Created, **204 No Content**. `res.ok` is true for all three — which is why `status === 200` is the wrong test.

3XX — LOOK ELSEWHERE

`fetch` follows these for you by default. You usually never see one.

4XX — YOU GOT IT WRONG

400 bad request, 401 not logged in, 403 not allowed, 404 no such thing, **429 too many**.

5XX — THEY GOT IT WRONG

500, 502, 503. Nothing you send will fix it, and retrying hard makes it worse.

Your code asks one question: `res.ok`.
The number is for the message.

THE LAYER, AND WHAT IS NOT IN IT

```
1 // api.js – LAYER 5. No document. No setState. No decision about the screen.
2
3 export async function searchTitles(query, { signal }) {
4   const url = new URL(ENDPOINT);
5   url.searchParams.set('q', query);          // NOT '&q=' + query
6
7   const res = await fetch(url, { signal });
8   if (!res.ok) throw new ApiError('status', `שגיאה ${res.status} מהקטלוג`, '');
9
10  let payload;
11  try { payload = await res.json(); }
12  catch { throw new ApiError('shape', 'JSON לא היה בתשובת הקטלוג', ''); }
13
14  return toSuggestions(payload);              // one normaliser, two sources
15 }
```

► It returns a collection, or it throws a **WHOLE HEBREW SENTENCE**. Never a `null` that means “work it out”.

SENDING SOMETHING: POST, AND THE ONE HEADER THAT MATTERS

```
1  const res = await fetch(url, {  
2    method: 'POST',  
3    headers: { 'content-type': 'application/json' },  
4    body: JSON.stringify({ title, number }), // a STRING, always –  
5                                              // the same rule as setItem  
6    signal,  
7  });  
8  if (!res.ok) throw new ApiError('status', '...', `HTTP ${res.status}`);
```

body IS A STRING

Exactly like
`localStorage.setItem`. Hand it
an object and it coerces.

THE HEADER IS NOT DECORATION

Without it the server guesses, and
most of them guess `text/plain`.

AND IT TRIGGERS A PREFLIGHT

A `content-type:`
`application/json` is not a
“simple” request. The browser asks
permission first. Act 5.

LIVE CODING · 1 OF 2

BUILD THE LAYER, AND BREAK IT ON PURPOSE

- 1 Write `searchTitles` against a real API. Watch it work.
- 2 Type the endpoint wrong. **Read the error out loud.**
- 3 DevTools › Network › throttle to **Slow 3G**. Type again.
- 4 Tick **Offline**. Type again.
- 5 Ask the room what the user just saw. **Nothing. Twice.**

THE MOST CONFUSING ERROR IN FRONT-END DEVELOPMENT

WHAT YOUR CODE GETS

```
TypeError: Failed to fetch
```

No status. No headers. No body. No reason.

WHAT ACTUALLY HAPPENED

- 1 the request left your machine
- 2 the server received it
- 3 the server answered
200
- 4 the browser
refused to hand it to you

The server said yes. Your browser said no.

THE SAME CODE, TWO SERVERS

he.wikipedia.org sends the header · data.gov.il does not

reload open

THE SAME CODE, TWO SERVERS

fetch he.wikipedia.org — sends the header fetch data.gov.il — does not reset

THE CODE, AND IT IS IDENTICAL FOR BOTH

```
try {
  const res = await fetch(url);
  const data = await res.json();
} catch (err) {
  // what lands here?
}
```

ACCESS-CONTROL-ALLOW-ORIGIN: *

—

NO SUCH HEADER

—

WHAT HAPPENED	ALLOWED ORIGIN	BLOCKED ORIGIN
the request left your machine	yes	yes
the server processed it	yes	yes
the server answered 200	yes	yes
the Network tab shows the row	yes	yes
your JavaScript received the answer	yes	NO
what your catch was given	nothing — it succeeded	TypeError: Failed to fetch

► Press the second button **WITH THE NETWORK TAB OPEN**. The row is there. The status is 200. The size is real.

WHY THE BROWSER DOES THIS AT ALL

THE RULE

A page may **send** a request anywhere. It may only **read** the answer if the answer says it may.

WHY

Otherwise any page you open could ask your bank for your statement — **using your cookies** — and read the reply.

SO

It is not aimed at you. You are standing in the way of something that is aimed at somebody worse.

► **CHECK BEFORE YOU BUILD.** Open the URL, then look for `access-control-allow-origin` in the Network tab. Two minutes.

THERE IS NO TIMEOUT, AND THERE NEVER HAS BEEN

a server that takes eight seconds, and four ways to react

reloadopen

THE SERVER TAKES 8.0 SECONDS TO ANSWER

server delay ☐ never answer at all

no timeoutAbortSignal.timeout(2000)timeout + one retrycancel by handreset

LIVE

waiting for 0.00 s

idle

THE TWO WAYS TO WRITE A TIMEOUT

// short, and it cannot leak a timer
const signal = AbortSignal.any([
 caller.signal,
 AbortSignal.timeout(2000),
]);
await fetch(url, { signal });

// the long way, which is what an assistant writes
const c = new AbortController();
const t = setTimeout(() => c.abort(), 2000);
try { await fetch(url, { signal: c.signal }); }
finally { clearTimeout(t); } // <- the line people leave out

STRATEGY

SERVER DELAY

THE USER WAITED

ATTEMPTS

HOW IT ENDED

► Run **NO TIMEOUT** at eight seconds first, and say nothing while it runs.

TWO REASONS TO STOP, ONE SIGNAL

WHAT AN ASSISTANT WRITES

```
1 const c = new AbortController();
2 const t = setTimeout(() => c.abort(), 4000);
3 const res = await fetch(url, { signal: c.signal });
4 // ...and the clearTimeout in a finally
5 // which is the line that gets left out
```

WHAT IT IS

```
1 const signal = AbortSignal.any([
2   caller.signal,           // AbortError
3   AbortSignal.timeout(4000), // TimeoutError
4 ]);
5 const res = await fetch(url, { signal });
```

- any forwards whichever fired first **WITH ITS REASON** — which is exactly what the `catch` needs.

THE SEARCH BOX THAT ANSWERS THE WRONG QUESTION

two boxes, the same server – and a counter for disagreements

↶ reload

open ↷

TYPE THE SAME WORD IN BOTH BOXES

fix on the right:

debounce + abort + token

abort only

token only

reset counters

NO DEBOUNCE, NO ABORT, NO TOKEN

search

requests sent: 0 · answers applied: 0 · disagreements: 0

—

DEBOUNCE + ABORT + TOKEN

search

requests sent: 0 · answers applied: 0 · disagreements: 0

—

THE FOUR LINES

```
let controller = null;           // 1 – one per box
let seq = 0;                     // 2 – a ticket per search

async function run(query) {
  controller?.abort();           // 3 – the previous one is not wanted
  controller = new AbortController();
  const ticket = ++seq;

  const results = await search(query, { signal: controller.signal });

  if (ticket !== seq) return;     // 4 – a newer search has started
  show(results);
}
```

► Type פפריקה slowly in the left box and say nothing. Let somebody notice.

FOUR LINES, AND EACH ONE STOPS SOMETHING ELSE

```
1  let controller = null;
2  let seq = 0;
3
4  async function run(query) {
5    controller?.abort();           // 1 - stops the NETWORK
6    controller = new AbortController();
7    const ticket = ++seq;         // 2 - a ticket
8
9    patch({ status: 'loading', error: null }); // 3 - BEFORE the await
10
11    try {
12      const results = await search(query, { signal: controller.signal });
13      if (ticket !== seq) return; // 4 - stops the WRITE
14      patch({ status: 'done', results });
15    } catch (error) {
16      if (error.name === 'AbortError') return; // an abort is not an error
17      patch({ status: 'error', error: sentence(error) });
18    }
19  }
```

“IF IT ABORTS, WHY DO I NEED THE NUMBER?”

`abort()` IS SYNCHRONOUS

It stops the request. Immediately, and for good.

THE `await` IS NOT

A response that had **already arrived and resolved** when the abort landed still runs its continuation.

AND SOME THINGS CANNOT BE CANCELLED

A third-party SDK. A library that takes no signal. The ticket is the **only** fix there.

The abort stops the work. The ticket decides whether the answer is still wanted.

RETRY, AND WHAT IT ACTUALLY COSTS

```
1 for (let n = 0; n <= retries; n++) {  
2   try {  
3     return await attemptOnce(signal);  
4   } catch (error) {  
5     if (error.name === 'AbortError') throw error;  
6     if (error instanceof ApiError) throw error;  
7     if (n === retries) throw asApiError(error);  
8     await sleep(300 * 2 ** n, signal);  
9   }  
10 }
```

- 1 timeout, dropped connection**
— retry. It might work.
- 2 any status**
— never. The server already answered.
- 3 abort**
— never, and hard. They asked you to stop.

A retry does not make a slow request fast. It makes a failing one take twice as long to fail.

LIVE CODING · 2 OF 2

FIX THE BOX, IN THE ORDER THE BUGS APPEAR

- 1 Count the requests in the Network tab. **Six.** Add the debounce. **One.**
- 2 Throttle to Slow 3G and type fast. **The wrong list appears.** Add the abort.
- 3 Type fast again. **The panel flashes red.** Add the AbortError line.
- 4 Add the ticket, and say out loud what it catches that the abort does not.

WEEK 6 NAMED FOUR STATES. TWO OF THEM START TODAY.

WEEKS 6 – 10

empty and **success**. Reading an array from memory is instant, and reading it from disk cannot fail halfway.

TODAY

loading and **error** finally have something to do — because a request takes time and can fail.

AND A FIFTH

idle. “Nobody has asked” is not “I asked and there was nothing”.

FOUR FAILURES, AND WHAT EACH ONE LOOKS LIKE TO A PERSON

WHAT HAPPENED	WHAT YOUR CODE SEES	WHAT THE USER MUST BE TOLD
bad URL → 404	a resolved response, <code>res.ok === false</code>	“the catalogue returned an error (404). It is not something you typed.”
their server broke → 500	the same, with a different number	the same sentence — and no retry button , because it will not help
slow network	nothing, for as long as it takes	a loading state , then “this is taking too long” and a way to try again
offline	<code>TypeError: Failed to fetch</code>	“check your connection” — and the offer of the local catalogue

► Two of these look **IDENTICAL** to a user with no error state: nothing happens, twice, for opposite reasons.

FIVE STATES, OR THIRTY-TWO COMBINATIONS

the same journey through four booleans and through one field

↶ reload ↷ open ↶

FIVE STATES, TWO WAYS TO KEEP THEM

idle loading **results** empty error the sequence a real user produces reset

FOUR BOOLEANS

```
let isLoading = false;
let isError  = false;
let isEmpty  = false;
let results  = [];
```

פפריקה מעושנת

פלפל שחור

כמון טחון

1 region visible — which is the only number that ever means anything

ONE FIELD, FIVE VALUES

```
state.search = { status, results, error };

view = status === 'loading' ? 'loading'
      : status === 'error'  ? 'error'
      : status === 'idle'   ? 'idle'
      : results.length === 0 ? 'empty'
      : 'results';
```

פפריקה מעושנת

פלפל שחור

כמון טחון

1 region visible — which is the only number that ever means anything

STEP

WHAT HAPPENED

FOUR BOOLEANS: REGIONS VISIBLE

ONE FIELD: REGIONS VISIBLE

► Press **THE SEQUENCE A REAL USER PRODUCES**. The left panel contradicts itself three times. **NOBODY WROTE THAT COMBINATION.**

FIVE STATES, ONE ATTRIBUTE, AND ONE DERIVATION

```
1 export function selectSearchView(state) {  
2   const { status, results } = state.search;  
3   if (status === 'loading') return 'loading';  
4   if (status === 'error') return 'error';  
5   if (status === 'idle') return 'idle';  
6   return results.length === 0 ? 'empty' : 'results'; // DERIVED  
7 }  
8  
9 // render.js – one value, four lines that all read it  
10 const view = selectSearchView(state);  
11 panel.dataset.state = view;  
12 document.querySelector('#api-loading').hidden = view !== 'loading';
```

► empty is **DERIVED**. loading is **STORED**. Nothing on the screen tells you whether a request is open.

FIVE FAULTS IN A NETWORK LAYER THE MACHINE WROTE

```
1  async function search(q) {  
2    const data = await (await fetch(URL + '?q=' + q)).json();  
3    setResults(data.results);  
4    setLoading(false);  
5    setEmpty(data.results.length === 0);  
6  }
```

1

no `res.ok`

2

no timeout

3

no catch at all

4

`isLoading` +
`isEmpty`

5

no abort, no ticket

THREE QUESTIONS BEFORE YOU GO

- 1 The server answered **404** with a valid JSON body. Did your `catch` run?
- 2 You typed six letters. **How many requests left the machine?**
- 3 How many lines does `js/app.js` gain today, and which file did you not open at all?

TODAY'S HANDS-ON, AND WHAT NEXT WEEK COSTS

10 MINUTES · WARM-UP

Five small functions, four of them wrong. All five are pieces of `js/api.js`. No git, no marks.

80 MINUTES · YOUR PROJECT

`js/api.js`, the five states, the search box. **Keep the Network tab open the whole time.**

BEFORE YOU LEAVE

Turn the wifi off and type. **If there is a sentence and a way forward, you are finished. If the page just sits there, you are not.**

► Next week: `js/api.js` is where a wrong guess about somebody else's response becomes a crash at runtime.

THE COMPILER WILL CATCH IT BEFORE YOU RUN.